Tarea 5: Complejidades

19 Febrero 2026

Erich Emmanuel Romero Mejía

1- Complejidad Constante ($O(1)$) · Aquí el tiempo de ejecución es independiente del tamaño de la entrada

Es el mas eficiente, ya que su tiempo de ejecución no aumenta con mayores entradas

Ejemplo: Verificar si un número es par o impar

2- Complejidad Logarítmica ($O(\log n)$) El tiempo aumenta de forma lenta conforme crece la entrada

Normalmente ocurre cuando el algoritmo divide el problema a la mitad en cada paso

Ideal para grandes conjuntos de datos

Ejemplo: Búsqueda binaria en una lista ordenada o búsqueda en árboles binarios

3- Complejidad Lineal ($O(n)$) Son algoritmos cuyo tiempo de ejecución crece en proporción directa al tamaño de la entrada (Se duplica la entrada y se duplicara el tiempo)

Típica de procesos que recorren los datos una sola vez

Ejemplo: Leer datos de una lista mediante un for

4- Complejidad Cuasilineal ($O(n \log n)$) El tiempo de ejecución aumenta ligeramente más rápido que de forma lineal respecto al tamaño de entrada

Típico y eficiente en algoritmos basados en el "divide y vencerás"

Si la entrada se duplica el tiempo estará debajo del doble o poco más arriba de él

Ejemplo: Algoritmos de Geometría: Encontrar el par de puntos mas cercanos en un plano

5- Complejidad Cuadrática ($O(n^2)$)

El tiempo de ejecución es el cuadrado de la entrada (n)

Común en bucles anidados donde cada uno recorre n elementos

Ejemplo: Matrices donde se requiere un for para i y otro anidado para j hasta terminar

6- Complejidad Exponencial ($O(2^n)$)

Describe algoritmos cuyo tiempo de ejecución o uso de memoria se duplica con cada elemento adicional en los datos de entrada

Ineficientes y necesariamente evitables en la mayoría de los casos

Ejemplo: Algoritmos recursivos de Fibonacci no optimizados

7- Complejidad Factorial ($O(n!)$)

Algoritmos de crecimiento máximo, siendo a veces imposibles de ejecutar con entradas pequeñas en computadoras modernas

Su crecimiento (como su nombre indica) es exponencial a sus datos de entrada

Ejemplo: Crear permutaciones de una lista